

Redukovanje šuma iz zvuka koristeći algoritam brze Furijeove transformacije

Profesor: prof. dr. Filip Marić
Asistent: Milan Mitreski
Student: Gordan Đurić (20/2022)

Beograd, Januar 2026

Apstrakt

Ovaj seminarski rad bavi se problemom redukovanja šuma iz digitalnih audio signala, koji može nastati kao posljedica oštećenja fajlova, tehničkih smetnji tokom prenosa, ograničenja opreme za snimanje ili drugih faktora tehničke prirode. Šum predstavlja jedan od ključnih izazova u obradi signala, jer značajno utiče na kvalitet i razumljivost zvuka. U radu se istražuje primjena algoritma brze Furijeove transformacije (FFT) kao osnovnog alata za analizu i obradu signala u frekventnom domenu, gdje je moguće jasnije identifikovati komponente šuma u odnosu na korisni signal.

Prije implementacije samog algoritma, razmatra se matematičko modelovanje jednokanalnog audio signala, kao i metode za prepoznavanje i karakterizaciju šuma u vremenskom i frekventnom domenu. Poseban akcenat stavljen je na analizu ponašanja šuma nakon primjene diskretne Furijeove transformacije, te na ograničenja klasičnih metoda frekventnog odsjecanja, koje se pokazuje neefikasnim pri obradi signala velike dužine ili sa promjenljivim spektralnim karakteristikama.

Kako bi se prevazišla navedena ograničenja, u radu se uvodi koncept kratkoročne Furijeove transformacije (STFT), koja omogućava lokalnu analizu signala u vremensko-frekventnom prostoru. Prikazano je zašto ovaj pristup predstavlja adekvatnije rješenje za uklanjanje šuma iz realnih audio zapisa. Na kraju, razmatraju se mogućnosti unapređenja osnovnog algoritma, kao i kratak pregled modernijih metoda uklanjanja šuma zasnovanih na tehnikama mašinskog učenja i optimizacije, uz poređenje njihovih prednosti i ograničenja.

Sadržaj

1	Uvod	1
2	Teorijska osnova algoritama	4
2.1	Diskretni vremenski signali - zapis u računar	4
2.2	Brza Furijeova transformacija (FFT)	5
2.3	Prozorne funkcije	6
2.4	Kratkoročna Furijeova transformacija (STFT)	7
3	Modelovanje šuma	8
3.1	Normalan šum	8
3.2	Pink šum	9
3.3	Impulsivan šum	9
4	Uklanjanje šuma	10
5	Implementacija protočne obrade	14
5.1	Implementacija kompleksne aritmetike	14
5.2	FFT implementacija	16
5.3	STFT implementacija	18
5.4	Audio I/O	23
6	Prednosti i ograničenja	31
7	Napredniji algoritmi	33
8	Zaključak	35

Uvod

Prije nego što se pristupi rješavanju razmatranog problema, neophodno je uspostaviti odgovarajuću matematičku osnovu. Iz tog razloga, uvodni dio rada posvećen je definisanju osnovnih pojmova iz teorije signala, kao i uspostavljanju veze između njih. Dokazi navedenih tvrdnji, kao i detaljniji teorijski rezultati, mogu se pronaći u literaturi [2].

Definicija 1.: Neka je $\mathcal{D}$ otvoren podskup prostora $\mathbb{R}^n$, a neka je $\mathcal{C}$ podskup prostora $\mathbb{C}^k$, za prirodne brojeve n, k . Preslikavanje $f : \mathcal{D} \rightarrow \mathcal{C}$ naziva se **signal**.

Na osnovu ove definicije, signal se može posmatrati kao matematičko preslikavanje koje zavisi od konačnog broja nezavisnih promjenljivih. U kontekstu obrade signala, od posebnog značaja su dimenzija domena, kao i struktura kodomena. U nastavku navodimo nekoliko karakterističnih primjera.

- U slučaju $n = 1$, signal zavisi od jedne promjenljive i može se interpretirati kao vremenski zavisan signal. Ako je pritom $k = 1$, tada se radi o **jednokanalnom audio signalu**, dok za $k \geq 2$ govorimo o **višekanalnim audio signalima**. Ovakvi signali biće predmet razmatranja u nastavku rada.
- Kada je $n = 2$, domen signala može se interpretirati kao dvodimenzionalna mreža tačaka, što omogućava predstavljanje signala u obliku slike. Ako je $k = 1$, tada govorimo o **sivtonskom signalu**, pri čemu se svaki piksel može predstaviti realnom vrijednošću iz intervala $[0, 1]$, odnosno kao konveksna kombinacija crne i bijele boje. Za $k \geq 2$ dobijaju se **višekanalni signali**, kakvi se javljaju kod kolor-slike.

Nakon što je uspostavljena osnovna intuicija o pojmu signala, u nastavku razmatramo osnovne operacije koje se nad signalima primjenjuju u svrhu njihove analize i obrade.

Posmatrajmo periodičan signal f , koji bez gubitka opštosti možemo restrikovati na interval $[-\pi, \pi]$. Takav signal može se predstaviti pomoću Furijeovog reda u obliku

$$f(t) = \frac{a_0}{2} + \sum_{k=1}^{\infty} (a_k \cos(kt) + b_k \sin(kt)),$$

gdje su koeficijenti dati izrazima

$$a_k = \frac{1}{2\pi} \int_{-\pi}^{\pi} f(t) \cos(kt) dt, \quad b_k = \frac{1}{2\pi} \int_{-\pi}^{\pi} f(t) \sin(kt) dt.$$

Ovim postupkom signal f se razlaže na sumu elementarnih sinusoidnih komponenti, pri čemu koeficijenti a_k i b_k mjere doprinos odgovarajućih frekvencija u ukupnom signalu. Intuitivno, ovaj proces se može posmatrati kao razlaganje složenog akorda na njegove osnovne tonove. Postavlja se, međutim, pitanje kako ovakav pristup uopštiti na neperiodične signale.

U tu svrhu razmatra se signal sa periodom T i njegovo periodično produženje f_T , definisano na intervalu $[-T, T]$. Tada važi

$$f = \lim_{T \rightarrow +\infty} f_T \quad (\text{t.p.t.}),$$

što omogućava prelaz sa Furijeovog reda na integralni oblik Furijeove transformacije. Na taj način dobija se predstavljanje

$$f(t) = \int_{\mathbb{R}} c(\alpha) e^{i\alpha t} d\alpha,$$

gdje je

$$c(\alpha) = \frac{1}{2\pi} \int_{\mathbb{R}} f(t) e^{-i\alpha t} dt.$$

Integral u navedenom izrazu naziva se **Furijeov integral**.

Definicija 2.: Funkcija $c(\alpha)$ naziva se **spektar** signala f .

U nastavku koristimo standardnu inženjersku konvenciju za Furijeovu transformaciju.

Definicija 3.: Neka je f neki signal nad domenom $\mathcal{D} \subset \mathbb{R}$ i kodomenom $\mathcal{C} \subset \mathbb{C}$. Pod pojmom **Furijeove transformacije** signala f smatramo preslikavanje $\mathcal{F}[f]$ definisano na idući način:

$$\mathcal{F}[f](\xi) := \int_{-\infty}^{\infty} f(t) e^{-i2\pi\xi t} dt$$

Dokazuje se postojanje **inverzne Furijeove transformacije**, i ona dolazi u obliku:

$$f(\xi) := \int_{-\infty}^{\infty} \mathcal{F}[f](t) e^{i2\pi\xi t} dt$$

Da bismo olakšali zapis, umjesto ove glomazne notacije sa $\mathcal{F}$ koristimo $\hat{f}$ da bismo obilježili Furijeovu transformaciju funkcije f .

Postavlja se pitanje, šta se postiže ovom transformacijom?

Kao što je bio slučaj u diskretnom slučaju, gledali smo uticaj signala $\cos(kx)$ i $\sin(kx)$, ovdje posmatramo uticaj svih mogućih frekvencija nad $\mathbb{R}$.

Da bismo ovo mogli sve da prebacimo u računar potrebno je da nekako diskretizujemo naše podatke, u pomoć nam ovdje dolazi teorema **Najkvist-Šanona**, čiji dokaz nalazimo u [2].

Teorema 1.: Neka je f ograničen signal sa najčešćom frekvencijom M . Tada broj uzorkovanja koje je potrebno da uzimamo da bi mogli vjerodostojno da rekonstruišemo signal mora biti veći od $2M$.

Što znači da možemo izvršiti restrikciju domena signala, umjesto da posmatramo podskupove od $\mathbb{R}^n$, možemo da posmatramo podskupove od $\mathbb{Z}^n$, ali samo u slučaju ograničenog signala.

Time smo omogućili zapis signala u digitalnom formatu, što omogućava zapis signala u računar. Umjesto da pišemo $f(nT)$, za neki prirodan broj n i neki period uzorkovanja T , pišemo $f[n]$, za signal f .

Sada sa ovim činjenicama, izvodimo iduću činjenicu. Diskretni signal x , koji je definisan u tačkama $0, 1, 2, \dots, N-1$, dodefinišemo tako da u ostalim tačkama ima vrijednost 0. Primjetimo da je period uzorkovanja u prvom signalu 1, pa važi $x[n] = x(n)$. Kako postoji Furijeova transformacija signal f , time možemo na ovakav signal da koristimo Rimanove sume kao aproksimaciju. No zbog Najkvist-Šanona, znamo da ako dovoljno veliko N odaberemo, da imamo garantovanu nejednakost. Tada kada vršimo Furijeovu transformaciju na tako definisan signal dobijamo:

$$\hat{x}(\xi) = \int_{\mathbb{R}} x(t) e^{-i2\pi\xi t} dt = \sum_{k=0}^{N-1} x[k] e^{-2\pi i \xi k}$$

kako je ξ slobodan parametar, biramo da uzorkujemo na tačkama $0, \frac{1}{N}, \frac{2}{N}, \dots, \frac{N-1}{N}$, obilježavamo $\hat{x}[k] = \hat{x}\left(\frac{k}{N}\right)$.

Pa smo dati oblik sveli na:

$$\hat{x}[k] = \sum_{n=0}^{N-1} x[n] e^{-\frac{2\pi i k n}{N}}$$

Ovaj diskretni oblik, nosi naziv **diskretna Furijeova transformacija (DFT)**.

Slično se dokazuje postojanje **inverznog** diskretnog oblika, i on se dobija pomoću ove formule.

$$x[k] = \frac{1}{N} \sum_{n=0}^{N-1} \hat{x}[n] e^{\frac{2\pi i k n}{N}}$$

Sada sa svim razvijenim pojmovima, spremni smo da nastavimo naše istraživanje dalje.

Teorijska osnova algoritama

2.1 Diskretni vremenski signali - zapis u računar

Naše detaljnije teorijsko izlaganje započinjemo sa izučavanjem diskretnih vremenskih signala. Kao što smo u našem prvom izlaganju rekli, ako nam je domen jednodimenzioni, tada možemo našu funkciju da promatramo kao neku koja zavisi od vremena, kao što bi bio audio. Jednostavnosti radi, ograničavamo naše istraživanje na jednokanalni audio format (kodomen je isto jednodimenzioni), iako bi se naša istraga analogno širila po svim koordinatama.



Figure 2.1: Primjer jednokanalnog audio formata

Jedno dodatno ograničenje koje sebi postavljamo, jednostavnosti radi, jeste da radimo sa WAV formatima, sa frekvencijom najvećom od 44kHz.

Postavlja se pitanje kako zapisujemo audio fajlove u računar? Odgovor na to pitanje leži u teoremi Najkvist-Šanona!

Naime svaki audio fajl kada se snima, vrši uzorkovanje nad određenim frekvencijama, i samim tim vjerodostojno rekonstruiše sam zvuk. WAV fajl se sastoji od niza binarnih blokova (chunk-ova), pri čemu su osnovni:

1. **RIFF header:**

- 4 bajta: identifikator "RIFF"
- 4 bajta: veličina ostatka fajla (`chunkSize`)
- 4 bajta: identifikator formata "WAVE"

2. **fmt chunk** (format podataka):

- 4 bajta: identifikator "fmt "

- 4 bajta: veličina podsekcije (`subchunk1Size`)
- 2 bajta: audio format (1 = PCM)
- 2 bajta: broj kanala (mono = 1, stereo = 2)
- 4 bajta: sample rate (Hz)
- 4 bajta: byte rate (`sampleRate` $\times$ `blockAlign`)
- 2 bajta: block align (broj bajtova po uzorku svih kanala)
- 2 bajta: bitna dubina po kanalu (npr. 16)

3. data chunk (zvukovni uzorci):

- 4 bajta: identifikator "data"
- 4 bajta: veličina audio podataka u bajtovima
- N bajtova: stvarni audio uzorci, po uzorku i po kanalu

Header se zapisuje prvi, kako bi softver mogao da pročita osnovne informacije o fajlu. Podaci se zapisuju u binarnom obliku, redom po uzorcima. Veličine chunk-ova i broj kanala omogućavaju precizno parsiranje fajla. WAV fajlovi su **jednostavni za parsiranje** i zato se često koriste u eksperimentima sa digitalnom obradom zvuka, iako nemaju kompresiju.

2.2 Brza Furijeova transformacija (FFT)

Početkom hladnog rata, Cooley i Tuckley su nezavisno došli do iste ideje, koju je Gauss još ranije koristio kao trik pri računanju, koja će se smatrati jednim od revolucionarnim algoritama svijeta, a to je algoritam "Brze Furijeove Transformacije".

Lema (Cooley & Tuckley): Neka je N neki stepen dvojke i neka je:

$$X_k = \sum_{n=0}^{N-1} x_n e^{-\frac{2\pi i}{N} kn}$$

tada se X_k može predstaviti na idući način:

$$X_k = E_k + e^{-\frac{2\pi i}{N} k} O_k$$

$$X_{k+\frac{N}{2}} = E_k - e^{-\frac{2\pi i}{N} k} O_k$$

gdje je E_k zbir po parnim članovima, a O_k po neparnim.

Ova lema je omogućila konstrukciju algoritma koji je $O(n \ln n)$ i samim tim vrši DFT u manje koraka. Dokaz ove leme je čisto algebarski i ne treba puno dovijanja da se dođe do traženog rezultata.

2.3 Prozorne funkcije

Za funkciju $f : \mathbb{R} \rightarrow \mathbb{R}$ kažemo da je prozorna, ako je dobro-definisana na nekom simetričnom segmentu, a van tog segmenta je 0

Prozorne funkcije se koriste u digitalnoj obradi signala kako bi se ograničio signal u vremenu i smanjili neželjeni efekti koji nastaju prilikom primjene Fourierove transformacije na konačne segmente signala. Pošto realni signali imaju ograničenu dužinu, njihovo direktno sječenje može dovesti do pojave spektralnog curenja.

Primjena prozorne funkcije podrazumijeva množenje ulaznog signala sa funkcijom koja ima glatke prelaze ka nuli na ivicama segmenta. Na taj način se ublažavaju nagle diskontinuiteti na granicama posmatranog vremenskog intervala, čime se poboljšava kvalitet frekvencijske analize.

Neka je $x[n]$ diskretni signal, a $w[n]$ prozorna funkcija dužine N . Prozorisani signal se dobija kao:

$$x_w[n] = x[n] \cdot w[n], \quad 0 \leq n < N$$

Različite prozorne funkcije imaju različite karakteristike u pogledu širine glavnog spektralnog lobusa i nivoa bočnih lobusa. Pravougaoni prozor ima najuži glavni lobus, ali i izraženo spektralno curenje. Hanning i Hamming prozori pružaju bolju potisnutost bočnih lobusa uz nešto širi glavni lobus, dok Blackman prozor dodatno smanjuje curenje po cenu slabije frekvencijske rezolucije.

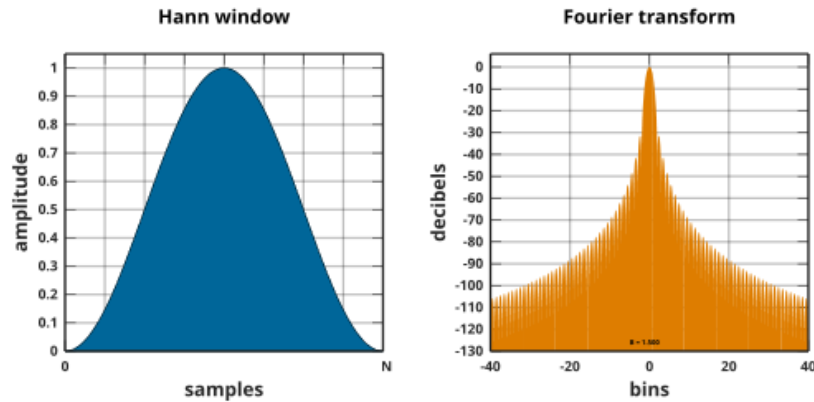


Figure 2.2: Hann prozorna funkcija

Izbor prozorne funkcije predstavlja kompromis između frekvencijske rezolucije i potiskivanja spektralnog curenja, a zavisi od prirode signala i cilja analize. Prozorne funkcije su neophodan deo algoritama kao što je STFT i imaju ključnu ulogu u vremensko-frekventnoj analizi signala.

2.4 Kratkoročna Furijeova transformacija (STFT)

Kratkoročna Furijeova transformacija (STFT) je metoda za analizu vremensko-frekventnog sadržaja signala. Za razliku od klasične Furijeove transformacije, koja daje samo globalni frekvencijski spektar, STFT omogućava praćenje promena frekvencija tokom vremena.

Osnovna ideja STFT algoritma je da se ulazni signal podeli na kratke, vremenski ograničene segmente. Svaki segment se množi sa prozornom funkcijom kako bi se smanjili rubni efekti, nakon čega se nad njim primenjuje Furijeova transformacija. Ovaj postupak se ponavlja duž celog signala, pri čemu se prozori često preklapaju.

Matematički, STFT diskretnog signala $x[n]$ definisana je kao:

$$X(m, k) = \sum_{n=0}^{N-1} x[n + mH] w[n] e^{-j2\pi kn/N}$$

gde je $w[n]$ prozorna funkcija dužine N , H korak pomjeranja prozora, m indeks vremenskog segmenta, a k indeks frekvencije.

Rezultat STFT algoritma je kompleksna matrica koja opisuje raspodjelu energije signala po vremenu i frekvencijama. Modul ove matrice se često prikazuje u obliku spektrograma. Izbor dužine prozora utiče na kompromis između vremenske i frekvencijske rezolucije: kraći prozor omogućava bolju vremensku lokalizaciju, dok duži prozor daje precizniju frekvencijsku analizu.

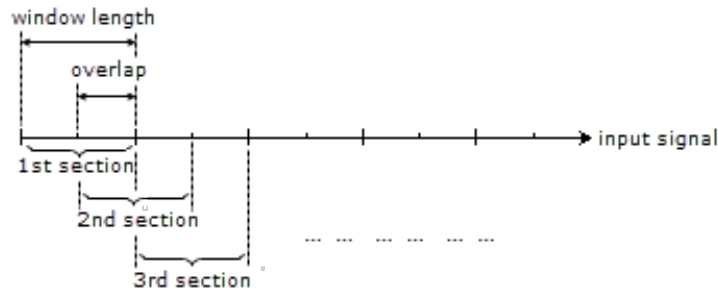


Figure 2.3: Okvirni rad STFT

Na osnovu opisa algoritma, znamo da je vremenska složenost algoritma $O(\frac{L}{H} N \ln N)$, gdje je L dužina unosa.

Modelovanje šuma

Šum predstavlja neželjenu komponentu signala koja nastaje kao posljedica fizičkih ograničenja sistema, spoljašnjih smetnji ili procesa mjerenja i prenosa signala. U digitalnoj obradi signala, modelovanje šuma omogućava simulaciju realnih uslova rada sistema i analizu otpornosti algoritama na smetnje. U ovom poglavlju razmatraju se najčešće korišćeni modeli šuma.

3.1 Normalan šum

Normalan ili Gaussov šum je najčešće korišćen model šuma u teoriji signala. Njegove amplitude prate normalnu (Gaussovu) raspodjelu sa srednjom vrednošću nula i određenom standardnom devijacijom. Ovaj tip šuma se često koristi za modelovanje termičkog šuma i drugih prirodnih izvora smetnji.

Matematički, normalan šum se opisuje funkcijom gustine verovatnoće:

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{x^2}{2\sigma^2}}$$

gde je σ standardna devijacija koja određuje intenzitet šuma. Normalan šum ima ravnomjernu raspodjelu energije po frekvencijama i često se naziva i bijelim šumom.

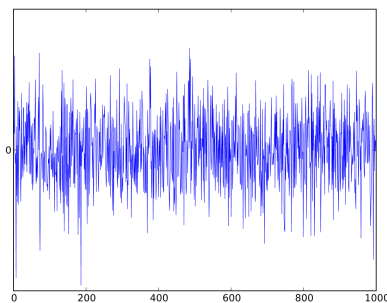


Figure 3.1: Bijeli šum

3.2 Pink šum

Pink šum je tip šuma kod kojeg gustina spektralne snage opada proporcionalno sa frekvencijom, odnosno prati zakon $1/f$. Za razliku od bijelog šuma, pink šum ima veću energiju u nižim frekvencijama, zbog čega se češće susreće u prirodnim i biološkim signalima.

Ovaj tip šuma se često koristi u audio inženjerstvu, naročito pri testiranju zvučnih sistema i perceptivnim analizama sluha, jer je njegova spektralna raspodjela bliža načinu na koji ljudsko uho percipira zvuk.

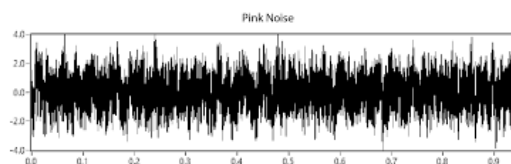


Figure 3.2: Pink šum

3.3 Impulsivan šum

Impulsivan šum se karakteriše rijetkim, ali izraženim amplitudnim skokovima u signalu. Ovi impulsi imaju kratko trajanje i znatno veće amplitude u poređenju sa osnovnim signalom. Impulsivan šum se javlja u komunikacionim sistemima, elektroenergetskim mrežama i digitalnim senzorima.

Zbog svoje nelinearne prirode, impulsivan šum može značajno degradirati kvalitet signala i često zahtjeva posebne tehnike filtriranja i detekcije. U modelovanju, impulsivan šum se obično opisuje kao nasumična pojava impulsa sa malom vjerovatnoćom pojavljivanja, ali velikom amplitudom.

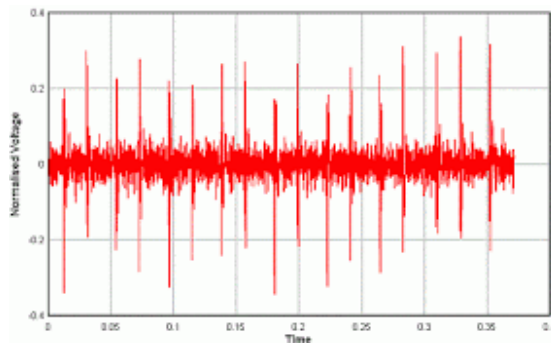


Figure 3.3: Impulsivni šum

Uklanjanje šuma

Prije nego krenemo sa opisom glavnog algoritma, krenimo sa intuicijom iza toga zašto baš koristimo Furijeovu transformaciju da uklonimo šum.

Kao što smo već rekli mi ćemo početni signal razložiti na njeogve frekvencije. Šum koji nastaje u pozadini on doprinosi našem audio signalu. Ako je šum koji nastaje pri niskim frekvencijama postupak uklanjanja šuma će teći na idući način:

Algorithm 1 Naivni algoritam

Require: Signal $x[n]$, granica τ

Ensure: Čist signal $\hat{x}[n]$

```
1:  $X[k] \leftarrow \text{FFT}(x[n])$ 
2: for  $k = 0$  to  $N - 1$  do
3:   if  $|X[k]| < \tau$  then
4:      $X[k] \leftarrow 0$ 
5:   end if
6: end for
7:  $\hat{x}[n] \leftarrow \text{IFFT}(X[k])$ 
8: return  $\hat{x}[n]$ 
```

Dakle algoritam je veoma jednostavan, zanemari sve frekvencije ispod neke granice i postavi te vrijednost koje su ispod granice da budu jednake nuli. Na taj način održavamo prvobitnu strukturu audio snimka, a svaki šum nestaje jer je njegov doprinos malen.

Primijetimo da ovdje postupak nije narušio složenost, jer unutrašnja petlja je linearne složenosti, dakle ovaj naivni algoritam radi u $O(n \ln n)$.

Opisani naivni algoritam ima intuitivnu i jednostavnu implementaciju, a njegova vremenska složenost ostaje nepromijenjena u odnosu na klasični FFT, odnosno $O(n \log n)$. Međutim, u praksi ovaj pristup pokazuje značajna ograničenja.

Prije svega, algoritam posmatra signal na globalnom nivou, bez ikakve vremenske lokalizacije. U realnim audio signalima karakteristike šuma se često mijenjaju tokom vremena, što ovaj pristup ne može da detektuje. Dodatno, kod određenih tipova šuma, poput bijelog šuma, energija šuma je ravnomjerno raspoređena po frekvencijama i često se preklapa sa spektrom korisnog signala, čime

se onemogućava jasno razdvajanje na osnovu amplitude.

Kako da popravimo trenutnu situaciju?

U praksi se često radi sa jednom modifikacijom STFT algoritma, zasnovana na ideji da je šum sam po sebi na neki način predvidljiv. Metoda se zasniva na procjeni spektra šuma i njegovom oduzimanju iz spektra svakog vremenskog okvira signala, uz rekonstrukciju signala metodom preklapanja i sabiranja.

Na početku se signal dijeli na vremenske okvire dužine N , koji se međusobno preklapaju za iznos definisan pomakom H (hop size). Ukupan broj okvira određuje se na osnovu dužine signala i vrijednosti pomaka. Za rekonstrukciju signala unaprijed se alociraju izlazni niz i pomoćni niz koji sadrži sume kvadrata prozorne funkcije.

Procjena spektra šuma vrši se na osnovu prvog vremenskog okvira signala, pod pretpostavkom da on sadrži dominantno šum. Signal u tom okviru se množi prozornom funkcijom i nad njim se primjenjuje FFT. Dobijeni kompleksni spektar se svodi na amplitudni spektar, pri čemu se fazna informacija zanemaruje. Ovaj spektar se koristi kao referentni model šuma tokom celog procesa obrade.

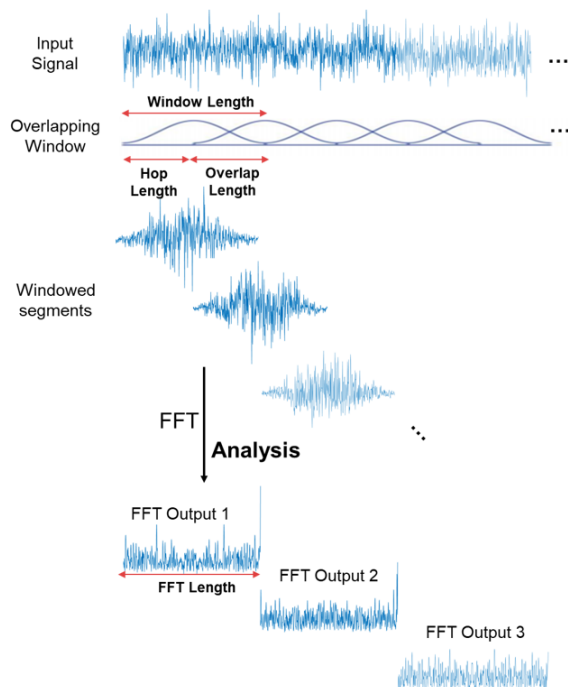


Figure 4.1: Grafički prikaz rada algoritma

U glavnoj petlji algoritma, svaki vremenski okvir signala se najpre prozori, a

zatim transformiše u frekvencijski domen pomoću FFT algoritma. Na dobijeni spektar primenjuje se jednostavna spektralna suptrakcija, pri čemu se amplituda spektra umanjuje za određeni procenat prethodno procenjenog spektra šuma. Negativne vrednosti se ograničavaju na nulu kako bi se izbegle nerealne amplitudne vrednosti. Fazna komponenta spektra se zadržava, čime se čuva vremenska struktura signala.

Nakon spektralne obrade, signal se vraća u vremenski domen primjenom IFFT. Očišćeni vremenski okvir se zatim kombinuje sa ostalim okvirima korišćenjem metode preklapanja i sabiranja (overlap-add). Svaki uzorak izlaznog signala predstavlja sumu preklapajućih prozornih segmenata, dok se paralelno akumulira suma kvadrata prozornih funkcija radi normalizacije.

U završnom koraku, izlazni signal se normalizuje djeljenjem sa sumom kvadrata prozora, čime se kompenzuje efekat preklapanja i obezbeđuje očuvanje amplitude signala. Rezultat je signal smanjenog nivoa šuma, rekonstruisan u vremenskom domenu.

Ovaj pristup predstavlja jednostavnu, ali efikasnu tehniku redukcije šuma i često se koristi kao osnova za naprednije metode obrade zvuka u vremensko-frekventnom domenu.

Opišimo sada ovaj proces u pseudo kodu.

Algorithm 2 STFT Redukcija šuma preko spektralne supstrakcije

Require: Signal $x[n]$, faktor uklanjanja α , prozorna funkcija $w[n]$ **Ensure:** Očišćen signal $\hat{x}[n]$

```
1:  $N \leftarrow 1024$ 
2:  $H \leftarrow N/2$ 
3:  $F \leftarrow \lceil |x|/H \rceil$ 
4: Inicijalizuj izlazeći signal  $y[n] \leftarrow 0$ 
5: Inicijalizuj prozorne sume  $s[n] \leftarrow 0$ 
6: Izdvoj prvi okvir  $x_0[n] = x[n] \cdot w[n]$ 
7:  $X_0[k] \leftarrow \text{FFT}(x_0[n])$ 
8: for  $k = 0$  to  $N - 1$  do
9:    $N[k] \leftarrow |X_0[k]|$ 
10: end for
11: for  $f = 0$  to  $F - 1$  do
12:   Izdvoji okvir  $x_f[n] = x[fH + n] \cdot w[n]$ 
13:    $X_f[k] \leftarrow \text{FFT}(x_f[n])$ 
14:   for  $k = 0$  to  $N - 1$  do
15:      $A \leftarrow |X_f[k]| - \alpha \cdot N[k]$ 
16:     if  $A < 0$  then
17:        $A \leftarrow 0$ 
18:     end if
19:      $\phi \leftarrow \arg(X_f[k])$ 
20:      $X_f[k] \leftarrow A \cdot e^{j\phi}$ 
21:   end for
22:    $\hat{x}_f[n] \leftarrow \text{IFFT}(X_f[k])$ 
23:   for  $n = 0$  to  $N - 1$  do
24:      $y[fH + n] \leftarrow y[fH + n] + \hat{x}_f[n] \cdot w[n]$ 
25:      $s[fH + n] \leftarrow s[fH + n] + w^2[n]$ 
26:   end for
27: end for
28: for  $n = 0$  to  $|x| - 1$  do
29:   if  $s[n] > 0$  then
30:      $y[n] \leftarrow y[n]/s[n]$ 
31:   end if
32: end for
33: return  $y[n]$ 
```

Implementacija protočne obrade

Sada kada smo prošli kroz sve potrebne korake, predjimo na implementaciju u programskom jeziku C++. Sve biblioteke su ručno kucane i ne zahtjevaju neku dodatnu biblioteku.

5.1 Implementacija kompleksne aritmetike

Za rad sa FFT je potrebno da implementiramo aritmetiku na kompleksnim brojevima, u tekstu ispod nalazi jedna jednostavna implementacija aritmetika kompleksnih brojeva!

Listing 5.1: Struktura kompleksnog broja, complex.hpp

```
1 #ifndef __COMPLEX_HPP
2 #define __COMPLEX_HPP
3
4 #include <cmath>
5 #include <iostream>
6
7 class Complex{
8     public:
9         Complex(): m_real(0.0f), m_imag(0.0f) {}
10        Complex(float x_mod, float y_angle, bool polar =
11            false): m_real(x_mod), m_imag(y_angle){
12            if (polar)
13            {
14                m_real = x_mod*std::cos(y_angle);
15                m_imag = x_mod*std::sin(y_angle);
16            }
17        }
18        ~Complex() {}
19
20        //functions for complex numbers
21        float module() const;
22        float angle() const;
23        float real() const;
24        float imag() const;
```

```

25
26 // operators
27 Complex operator+(const Complex& other) const;
28 Complex operator-(const Complex& other) const;
29 Complex operator*(const Complex& other) const;
30 Complex operator*(const float skalar) const;
31 Complex operator/(const Complex& other) const;
32 Complex operator~() const;
33 bool operator==(const Complex& other) const;
34
35 friend std::ostream& operator<<(std::ostream& out,
36     const Complex &obj);
37 friend std::istream& operator>>(std::istream& in,
38     Complex &obj);
39
40 friend Complex round_complex(Complex c);
41
42 private:
43     float m_real, m_imag;
44 };
45 #endif

```

Listing 5.2: Implementacija metoda, complex.cpp

```

1 #include <cmath>
2 #include "complex.hpp"
3
4 float Complex::module() const
5 {
6     return std::sqrt(this->m_real*this->m_real + this->
7         m_imag*this->m_imag);
8 }
9
10 float Complex::real() const
11 {
12     return m_real;
13 }
14
15 float Complex::imag() const
16 {
17     return m_imag;
18 }
19
20 Complex Complex::operator+(const Complex& other ) const {
21     return Complex(other.m_real+this->m_real, other.m_imag+
22         this->m_imag);
23 }
24
25 Complex Complex::operator-(const Complex& other ) const {

```

```

24     return Complex(-other.m_real+this->m_real,-other.m_imag+
25         this->m_imag);
26 }
27 Complex Complex::operator*(const Complex& other) const {
28     return Complex(this->m_real*other.m_real-this->m_imag*
29         other.m_imag,this->m_real*other.m_imag+other.m_real*
30         this->m_imag);
31 }
32 Complex Complex::operator*(const float skalar) const {
33     return Complex(this->m_real*skalar,this->m_imag*skalar);
34 }
35 Complex Complex::operator/(const Complex& other) const {
36     return (*this)*(~other)*(Complex(1/pow(other.module(),2)
37         ,0));
38 }
39 Complex Complex::operator~() const {
40     return Complex(this->m_real,-this->m_imag);
41 }
42
43
44 std::ostream &operator<<(std::ostream &out, const Complex &
45     obj)
46 {
47     out << obj.m_real << " + " << obj.m_imag << "i";
48     return out;
49 }
50 std::istream &operator>>(std::istream &in, Complex &obj)
51 {
52     in >> obj.m_real >> obj.m_imag;
53     return in;
54 }
55
56 Complex round_complex(Complex c)
57 {
58     float _base = 10000.0f;
59     c.m_imag = roundf64(c.m_imag*_base)/_base;
60     c.m_real = roundf64(c.m_real*_base)/_base;
61     return c;
62 }

```

5.2 FFT implementacija

Listing 5.3: Implementacija FFT-a, utils.hpp

```

1  #ifndef __UTILS_HPP
2  #define __UTILS_HPP
3
4  #include <vector>
5  #include <climits>
6
7  #include "complex.hpp"
8
9  /**
10 * Ovaj header file sadrzi sve potrebne funkcije za
11 * funkcionisanje fft-a
12 * kao i neka jednostavnija izracunavanja, potrebna za dio
13 * koji koristi masinsko ucenje
14 */
15 typedef std::vector<Complex> ComplexVector; // omogucava
16 jezgrovitosti zapis
17
18 /**
19 * Pomocne funkcije
20 */
21 unsigned getNearestPower(int n, int base){
22     unsigned tmp = 1;
23     while(tmp <= n)
24         tmp *= base;
25     return tmp;
26 }
27
28
29
30 /**
31 * 1D FFT!
32 */
33
34 void fft(const ComplexVector& a, int start_a, const
35 ComplexVector& w, ComplexVector& rezultat, int
36 start_rezultat, int korak){
37     int n = a.size()/korak;
38     if (n == 1)
39     {
40         rezultat[start_rezultat] = a[start_a];
41         return ;
42     }
43     fft(a, start_a, w, rezultat, start_rezultat, korak*2);

```

```

43     fft(a,start_a+korak,w,rezultat,start_rezultat+n/2,korak
      *2);
44
45     for (int k = 0; k < n/2; k++)
46     {
47         Complex r1 = rezultat[start_rezultat+k];
48         Complex r2 = rezultat[start_rezultat+(k+n/2)];
49
50         rezultat[start_rezultat+k] = round_complex(r1 + w[k*
      korak]*r2);
51         rezultat[start_rezultat+(k+n/2)] = round_complex(r1
      - w[k*korak]*r2);
52     }
53 }
54
55 ComplexVector fft(const ComplexVector& a){
56     int n = a.size();
57     ComplexVector w(n/2);
58     for (int i = 0; i < w.size(); i++)
59         w[i] = Complex(1,2*i*M_PI/n,true);
60     ComplexVector rezultat(n);
61     fft(a,0,w,rezultat,0,1);
62     return rezultat;
63 }
64
65 ComplexVector ifft(const ComplexVector& a){
66     int n = a.size();
67     ComplexVector w(n/2);
68     for (int i = 0; i < w.size(); i++)
69         w[i] = Complex(1,-2*i*M_PI/n,true);
70     ComplexVector rezultat(n);
71     fft(a,0,w,rezultat,0,1);
72     for(int i = 0;i < rezultat.size();i++)
73         rezultat[i] = round_complex(rezultat[i]*(1/((float)n
      ))));
74     return rezultat;
75 }
76
77
78
79 #endif

```

5.3 STFT implementacija

Listing 5.4: STFT implementacija

```

1 #ifndef __REMOVE_HPP
2 #define __REMOVE_HPP

```

```

3
4 #include <vector>
5
6 #include "audio.hpp"
7 #include "utils.hpp"
8 #include "complex.hpp"
9 #include "wavheader.hpp"
10 #include "wfunctions.hpp"
11
12 #include "types.hpp"
13
14 std::vector<float> removeNoiseSTFT(const std::vector<float>&
    samples, const float procentage_removal, const
    WINDOW_FUNCTION wf = WINDOW_FUNCTION::HANN){
15     int N = 1024;
16     int hop = N/2;
17
18     std::vector<float> window;
19     if (wf == WINDOW_FUNCTION::NONE)
20     {
21         window = noneWindow(N);
22     }else if (wf == WINDOW_FUNCTION::HAMMING)
23     {
24         window = hammingWindow(N);
25     }else if (wf == WINDOW_FUNCTION::FLAT_TOP)
26     {
27         window = flatTopWindow(N);
28     }else if (wf == WINDOW_FUNCTION::HANN)
29     {
30         window = hannWindow(N);
31     }else if (wf == WINDOW_FUNCTION::PARZEN)
32     {
33         window = parzenWindow(N);
34     }else if (wf == WINDOW_FUNCTION::WELCH)
35     {
36         window = welchWindow(N);
37     }
38
39     int numFrames = (samples.size() + hop - 1) / hop;
40     std::vector<float> output(samples.size() + N, 0.0f);
41     std::vector<float> windowSums(output.size(), 0.0f);
42
43     ComplexVector noiseSpectrum(N);
44     {
45         ComplexVector firstFrame(N);
46         for (int i = 0; i < N; i++) {
47             float x = (i < samples.size()) ? samples[i] *
                window[i] : 0.0f;
48             firstFrame[i] = Complex(x, 0.0f);
49         }

```

```

50     noiseSpectrum = fft(firstFrame);
51     for (auto& c : noiseSpectrum) c = Complex(c.module()
52         , 0.0f);
53 }
54
55 for (int f = 0; f < numFrames; f++) {
56     ComplexVector frame(N);
57     for (int i = 0; i < N; i++) {
58         int idx = f * hop + i;
59         float x = (idx < samples.size()) ? samples[idx]
60             * window[i] : 0.0f;
61         frame[i] = Complex(x, 0.0f);
62     }
63
64     // FFT
65     ComplexVector spectrum = fft(frame);
66
67     // Spectral subtraction (simple)
68     for (int k = 0; k < N; k++) {
69         float mag = spectrum[k].module() -
70             procentage_removal*noiseSpectrum[k].real();
71         if (mag < 0.0f) mag = 0.0f;
72         float phase = std::atan2(spectrum[k].imag(),
73             spectrum[k].real());
74         spectrum[k] = Complex(mag * std::cos(phase), mag
75             * std::sin(phase));
76     }
77
78     // IFFT
79     ComplexVector cleanedFrame = ifft(spectrum);
80
81     // Overlap-add
82     for (int i = 0; i < N; i++) {
83         int idx = f * hop + i;
84         if (idx < output.size()) {
85             output[idx] += cleanedFrame[i].real() *
86                 window[i];
87             windowSums[idx] += window[i] * window[i];
88         }
89     }
90 }
91
92 for (size_t i = 0; i < samples.size(); i++) {
93     if (windowSums[i] > 0.0f)
94         output[i] /= windowSums[i];
95 }
96
97 output.resize(samples.size());
98 return output;
99

```

```

94 }
95
96 #endif

```

Napomenimo ovdje da smo u zasebnom fajlu `types.hpp` deklarirali enum `WINDOW_FUNCTION`, zarad obezbjeđivanja modularnije upotrebe samog koda, dok same definicije funkcija koje se koriste nalaze se u zaglavlju `wfunctions.hpp`

Listing 5.5: `wfunctions.hpp`

```

1  #ifndef __WFUNCTIONS_HPP
2  #define __WFUNCTIONS_HPP
3
4  #include <cmath>
5  #include <vector>
6
7  std::vector<float> noneWindow(int N) {
8      std::vector<float> w(N, 1.0f);
9      return w;
10 }
11
12 std::vector<float> hannWindow(int N) {
13     std::vector<float> w(N);
14     for (int i = 0; i < N; i++)
15     {
16         w[i] = 0.5f * (1.0f - std::cos(2.0f * M_PI * i / (N
17             - 1)));
18     }
19     return w;
20 }
21
22 std::vector<float> hammingWindow(int N) {
23     std::vector<float> w(N);
24     for (int i = 0; i < N; i++) {
25         w[i] = 0.54f - 0.46f * std::cos(2.0f * M_PI * i / (N
26             - 1));
27     }
28     return w;
29 }
30
31 std::vector<float> flatTopWindow(int N) {
32     std::vector<float> w(N);
33     for (int i = 0; i < N; i++) {
34         float x = 2.0f * M_PI * i / (N - 1);
35         w[i] = 0.21557895f
36             - 0.41663158f * std::cos(x)
37             + 0.277263158f * std::cos(2.0f * x)
38             - 0.083578947f * std::cos(3.0f * x)
39             + 0.006947368f * std::cos(4.0f * x);
40     }
41     return w;
42 }

```



```

40 }
41
42
43 std::vector<float> welchWindow(int N) {
44     std::vector<float> w(N);
45     float half = (N - 1) / 2.0f;
46     for (int i = 0; i < N; i++) {
47         float n = (i - half) / half;
48         w[i] = 1.0f - n * n;
49     }
50     return w;
51 }
52
53
54 std::vector<float> parzenWindow(int N) {
55     std::vector<float> w(N);
56     float half = (N - 1) / 2.0f;
57
58     for (int i = 0; i < N; i++) {
59         float n = std::abs((i - half) / half);
60
61         if (n <= 0.5f) {
62             w[i] = 1.0f - 6.0f * n * n + 6.0f * n * n * n;
63         } else if (n <= 1.0f) {
64             float t = 1.0f - n;
65             w[i] = 2.0f * t * t * t;
66         } else {
67             w[i] = 0.0f;
68         }
69     }
70     return w;
71 }
72
73 #endif

```

Listing 5.6: types.hpp

```

1 #ifndef __TYPES_HPP
2 #define __TYPES_HPP
3
4 enum class WINDOW_FUNCTION{
5     NONE = 0,
6     HANN = 1,
7     WELCH = 2,
8     PARZEN = 3,
9     HAMMING = 4,
10    FLAT_TOP = 5
11 };
12
13

```

```

14 enum class NOISE_TYPE{
15     NORMAL = 0,
16     PINK = 1,
17     IMPULSIVE = 2
18 };
19
20
21 #endif

```

Na ovom mjestu naglašavamo malu napomenu, u idućoj skeicji opisujemo i generaciju šuma, samim tim bilo je potrebno napraviti distinkciju između šumova, otuda enum NOISE_TYPE.

5.4 Audio I/O

Listing 5.7: wavheader.hpp

```

1  #ifndef __WAVHEADER_HPP
2  #define __WAVHEADER_HPP
3
4  #include <vector>
5  #include <stdint>
6
7  #pragma pack(push, 1)
8  struct WAVHeader{
9      char riff[4];
10     uint32_t chunkSize;
11     char wave[4];
12     char fmt[4];
13     uint32_t subchunk1Size;
14     uint16_t audioFormat;
15     uint16_t numChannels;
16     uint32_t sampleRate;
17     uint32_t byteRate;
18     uint16_t blockAlign;
19     uint16_t bitsPerSample;
20     char data[4];
21     uint32_t dataSize;
22 };
23 struct AudioBuffer{
24     int sampleRate;
25     int channels;
26     std::vector<float> samples;
27 };
28 #pragma pack(pop)
29 #endif

```

Listing 5.8: Čitanje i upisivanje WAV fajlova

```

1 #ifndef __AUDIO_HPP
2 #define __AUDIO_HPP
3
4 #include <random>
5 #include <vector>
6 #include <stdint>
7 #include <cstring>
8 #include <fstream>
9 #include <algorithm>
10 #include <stdexcept>
11
12
13 #include "complex.hpp"
14 #include "wavheader.hpp"
15
16 #include "types.hpp"
17
18 AudioBuffer loadWAV(const std::string& filename) {
19     std::ifstream file(filename, std::ios::binary);
20     if (!file)
21         throw std::runtime_error("Cannot open WAV");
22
23     char id[4];
24     uint32_t size;
25
26     file.read(id, 4);
27     if (std::strncmp(id, "RIFF", 4))
28         throw std::runtime_error("Not RIFF");
29
30     file.read(reinterpret_cast<char*>(&size), 4);
31
32     file.read(id, 4);
33     if (std::strncmp(id, "WAVE", 4))
34         throw std::runtime_error("Not WAVE");
35
36     uint16_t audioFormat = 0;
37     uint16_t channels = 0;
38     uint32_t sampleRate = 0;
39     uint16_t bitsPerSample = 0;
40     uint32_t dataSize = 0;
41
42
43     while (file.read(id, 4)) {
44         file.read(reinterpret_cast<char*>(&size), 4);
45
46         if (!std::strncmp(id, "fmt ", 4)) {
47             file.read(reinterpret_cast<char*>(&audioFormat),
48                 2);

```

```

48         file.read(reinterpret_cast<char*>(&channels), 2)
49         ;
50         file.read(reinterpret_cast<char*>(&sampleRate),
51         4);
52         file.seekg(6, std::ios::cur);
53         file.read(reinterpret_cast<char*>(&bitsPerSample
54         ), 2);
55         file.seekg(size - 16, std::ios::cur);
56     }
57     else if (!std::strncmp(id, "data", 4)) {
58         dataSize = size;
59         break;
60     }
61     else {
62         file.seekg(size, std::ios::cur);
63     }
64 }
65
66 if (audioFormat != 1 || bitsPerSample != 16)
67     throw std::runtime_error("Unsupported WAV");
68
69 std::vector<int16_t> raw(dataSize / 2);
70 file.read(reinterpret_cast<char*>(raw.data()), dataSize)
71 ;
72
73 AudioBuffer audio;
74 audio.sampleRate = sampleRate;
75 audio.channels = channels;
76 audio.samples.resize(raw.size());
77
78 for (size_t i = 0; i < raw.size(); i++)
79     audio.samples[i] = raw[i] / 32768.0f;
80
81 return audio;
82 }
83
84 std::vector<float> toMono(const AudioBuffer& buffer){
85     if(buffer.channels == 1)
86         return buffer.samples;
87     std::vector<float> mono;
88     mono.reserve(buffer.samples.size()/buffer.channels);
89     for (size_t i = 0; i < buffer.samples.size(); i+=buffer.
90     channels)
91     {
92         float sum = 0.0f;
93         for (int ch = 0; ch < buffer.channels; ch++)
94         {
95             sum += buffer.samples[i+ch];
96         }
97         mono.push_back(sum/buffer.channels);
98     }
99 }

```

```

93     }
94     return mono;
95 }
96
97 void writeWAV(const std::string& filename, const std::vector
<float>& samples, int sampleRate, int channels=1){
98     std::ofstream file(filename, std::ios::binary);
99     if (!file)
100     {
101         throw std::runtime_error("Cannot open output WAV
file");
102     }
103
104     std::vector<int16_t> pcm(samples.size());
105     for (size_t i = 0; i < samples.size(); i++) {
106         double x = std::clamp(static_cast<double>(samples[i
]), -1.0, 1.0);
107         pcm[i] = static_cast<int16_t>(x * 32767.0);
108     }
109
110     WAVHeader header;
111
112     std::memcpy(header.riff, "RIFF", 4);
113     std::memcpy(header.wave, "WAVE", 4);
114     std::memcpy(header.fmt, "fmt ", 4);
115     std::memcpy(header.data, "data", 4);
116
117     header.subchunk1Size = 16;
118     header.audioFormat = 1;
119
120     header.bitsPerSample = 16;
121
122     header.numChannels = channels;
123     header.sampleRate = sampleRate;
124     header.blockAlign = channels * sizeof(int16_t);
125     header.byteRate = sampleRate * header.blockAlign;
126     header.dataSize = pcm.size() * sizeof(int16_t);
127     header.chunkSize = 36 + header.dataSize;
128
129     file.write(reinterpret_cast<char*>(&header), sizeof(
WAVHeader));
130     file.write(reinterpret_cast<char*>(pcm.data()), header.
dataSize);
131 }
132
133 std::vector<float> addNoise(const std::vector<float>&
samples,
134                             const float stddev,
135                             const NOISE_TYPE noise =
NOISE_TYPE::NORMAL)

```

```

136 {
137     std::random_device rd;
138     std::mt19937 gen(rd());
139
140     std::normal_distribution<float> normalDist(0.0f, stddev)
141     ;
142     std::uniform_real_distribution<float> uniformDist(0.0f,
143     1.0f);
144
145     // State for pink noise (simple Voss-style filter)
146     float pinkState = 0.0f;
147
148     std::vector<float> noisy;
149     noisy.reserve(samples.size());
150
151     for (auto s : samples) {
152         float noiseSample = 0.0f;
153
154         if (noise == NOISE_TYPE::NORMAL)
155         {
156             noiseSample = normalDist(gen);
157         } else if (noise == NOISE_TYPE::PINK)
158         {
159             float white = normalDist(gen);
160             pinkState = 0.98f * pinkState + 0.02f * white;
161             noiseSample = pinkState;
162         } else if (noise == NOISE_TYPE::IMPULSIVE)
163         {
164             if (uniformDist(gen) < 0.01f) {
165                 noiseSample = normalDist(gen) * 5.0f;
166             } else {
167                 noiseSample = 0.0f;
168             }
169         }
170
171         float n = s + noiseSample;
172         if (n > 1.0f) n = 1.0f;
173         if (n < -1.0f) n = -1.0f;
174
175         noisy.push_back(n);
176     }
177
178     return noisy;
179 }
180 #endif

```

Dakle upotreba na osnovu ovog izlaganja koda je iduća:

učitaj audio, loadWAV → pretvori u mono audio, toMono
→ (dodaj šum, addNoise) → očisti audio, removeNoiseSTFT

U idućem isječku napravili smo protočnu obradu koja prolazi kroz sve šumove, kao i prozorne funkcije nama dostupne!

Listing 5.9: main.cpp

```
1 #include <iostream>
2
3 #include "utils.hpp"
4 #include "audio.hpp"
5 #include "complex.hpp"
6
7 #include "types.hpp"
8 #include "remove.hpp"
9
10 void clean_all(std::vector<float>& normalNoise, const std::
    vector<float>& pinkNoise, const std::vector<float>&
    impulsiveNoise,
11             const WINDOW_FUNCTION wf, const float
    thresh_procentage[5],
12             const unsigned sampleRate, const unsigned
    channels = 1){
13
14     std::string applied_function;
15     switch (wf)
16     {
17     case WINDOW_FUNCTION::NONE:
18         std::cout << "Applying no window function" << std::
            endl;
19         applied_function = "_none_";
20         break;
21     case WINDOW_FUNCTION::FLAT_TOP:
22         std::cout << "Applying FLAT_TOP function" << std::
            endl;
23         applied_function = "_flat_";
24         break;
25     case WINDOW_FUNCTION::HAMMING:
26         std::cout << "Applying the Hamming window function"
            << std::endl;
27         applied_function = "_hamming_";
28         break;
29     case WINDOW_FUNCTION::HANN:
30         std::cout << "Applying the Hann window function" <<
            std::endl;
31         applied_function = "_hann_";
32         break;
```

```

33     case WINDOW_FUNCTION::PARZEN:
34         std::cout << "Applying the Parzen function" << std::
            endl;
35         applied_function = "_parzen_";
36         break;
37     case WINDOW_FUNCTION::WELCH:
38         std::cout << "Applying the Welch function" << std::
            endl;
39         applied_function = "_welch_";
40         break;
41     default:
42         break;
43 }
44
45
46 for (int i = 0; i < 5; i++)
47 {
48     std::string thresh_str = std::to_string(
        thresh_procentage[i]);
49     std::string message = "denoising... (factor: " +
        thresh_str + ")";
50
51     std::cout << "Normal " << message << std::endl;
52     std::vector<float> denoiseNormal = removeNoiseSTFT(
        normalNoise, thresh_procentage[i], wf);
53     writeWAV("data/cleaned/normal/rusija" +
        applied_function + thresh_str + ".wav",
        denoiseNormal, sampleRate, channels);
54
55     std::cout << "Pink " << message << std::endl;
56     std::vector<float> denoisePink = removeNoiseSTFT(
        pinkNoise, thresh_procentage[i], wf);
57     writeWAV("data/cleaned/pink/rusija" +
        applied_function + thresh_str + ".wav",
        denoisePink, sampleRate, channels);
58
59     std::cout << "Impulsive " << message << std::endl;
60     std::vector<float> denoiseImpulsive =
        removeNoiseSTFT(impulsiveNoise, thresh_procentage[
        i], wf);
61     writeWAV("data/cleaned/impulsive/rusija" +
        applied_function + thresh_str + ".wav",
        denoiseImpulsive, sampleRate, channels);
62 }
63
64 }
65
66 int main() {
67     auto a = loadWAV("data/src/rusija.wav");
68     std::cout << "Loaded in the audio..." << std::endl;

```



```

69     std::cout << "Converting to MONO audio" << std::endl;
70     std::vector<float> monoAudio = toMono(a);
71     writeWAV("data/rusija_mono.wav", monoAudio, a.sampleRate
72             , 1);
73     std::cout << "Adding normal(white) noise..." << std::
74             endl;
75     std::vector<float> normal_noise = addNoise(monoAudio
76             , 0.2, NOISE_TYPE::NORMAL);
77     writeWAV("data/sample/normal/rusija.wav", normal_noise,
78             a.sampleRate, 1);
79     std::cout << "Adding pink noise..." << std::endl;
80     std::vector<float> pink_noise = addNoise(monoAudio, 0.2f,
81             NOISE_TYPE::PINK);
82     writeWAV("data/sample/pink/rusija.wav", pink_noise, a.
83             sampleRate, 1);
84     std::cout << "Adding impulsive noise..." << std::endl;
85     std::vector<float> imp_noise = addNoise(monoAudio, 0.2f,
86             NOISE_TYPE::IMPULSIVE);
87     writeWAV("data/sample/impulsive/rusija.wav", imp_noise,
88             a.sampleRate, 1);
89
90     float thresh_procentages[5] = {0.2f, 0.5f, 1.0f, 1.5f,
91             2.0f};
92
93     clean_all(normal_noise, pink_noise, imp_noise,
94             WINDOW_FUNCTION::FLAT_TOP, thresh_procentages, a.
95             sampleRate);
96     clean_all(normal_noise, pink_noise, imp_noise,
97             WINDOW_FUNCTION::HAMMING, thresh_procentages, a.
98             sampleRate);
99     clean_all(normal_noise, pink_noise, imp_noise,
100            WINDOW_FUNCTION::HANN, thresh_procentages, a.sampleRate
101            );
102     clean_all(normal_noise, pink_noise, imp_noise,
103            WINDOW_FUNCTION::NONE, thresh_procentages, a.sampleRate
104            );
105     clean_all(normal_noise, pink_noise, imp_noise,
106            WINDOW_FUNCTION::WELCH, thresh_procentages, a.
107            sampleRate);
108     clean_all(normal_noise, pink_noise, imp_noise,
109            WINDOW_FUNCTION::PARZEN, thresh_procentages, a.
110            sampleRate);
111
112     return 0;
113 }

```

Prednosti i ograničenja

Upotreba STFT-a i spektralne supstrakcije za redukciju šuma u audio signalima donosi niz prednosti, ali i određena ograničenja koja je važno razumjeti.

Prednosti:

- **Lokalna vremensko-frekventna analiza:** STFT omogućava praćenje promjena u spektru signala tokom vremena, što čini metodu pogodnom za obradu nelinearnih i ne-stacionarnih šumova.
- **Jednostavna implementacija:** Korišćenjem FFT-a i overlap-add rekonstrukcije moguće je efikasno implementirati algoritam bez potrebe za složenim numeričkim metodama.
- **Kontrolisano uklanjanje šuma:** Parametar spektralne supstrakcije (faktor uklanjanja) omogućava balans između uklanjanja šuma i očuvanja korisnog signala.

Ograničenja:

- **Artefakti u vremenskom domenu:** Jedan od najčešćih neželjenih efekata je tzv. *musical noise* ili muzički šum, koji se manifestuje kao kratki, tonovski zvukovi niske amplitude. Oni nastaju zbog nesigurnosti u procjeni šuma i naglim promjenama spektra između susjednih vremenskih okvira.
- **Potencijalna degradacija signala:** Prekomjerno oduzimanje spektra šuma može ukloniti dijelove korisnog signala, naročito u slučajevima kada se šum preklapa sa frekvencijama korisnog signala.
- **Ograničenja rezolucije:** Dužina prozora i hop size utiču na kompromis između vremenske i frekvencijske rezolucije. Kratki prozor poboljšava vremensku lokalizaciju, ali smanjuje preciznost frekvencijskog spektrograma, i obrnuto.

U praksi, da bi se smanjili artefakti i muzički šum, često se primjenjuju dodatne tehnike poput adaptivne procjene spektra šuma, Wiener filtracije ili soft-masking metoda. Ove metode pomažu u očuvanju prirodnog zvuka signala dok se uklanja neželjeni šum.

Zaključno, STFT i spektralna supstrakcija predstavljaju efikasnu i jednostavnu tehniku redukcije šuma, ali je nužno biti svjestan njenih ograničenja i pažljivo birati parametre algoritma.

Napredniji algoritmi

Pored osnovne metode spektralne supstrakcije zasnovane na STFT-u, u savremenoj obradi zvuka koriste se i napredniji algoritmi koji omogućavaju efikasnije i preciznije uklanjanje šuma. Ovi pristupi dijele se uglavnom u dvije grupe: poboljšanja tradicionalnih STFT metoda i algoritme bazirane na mašinskom učenju.

Napredne STFT metode

Napredniji algoritmi zasnovani na STFT-u obično uključuju adaptivnu procjenu šuma i sofisticirane tehnike maskiranja. Primjeri uključuju:

- **Wiener filtracija u vremensko-frekventnom domenu:** koristi procjenu spektralne snage signala i šuma za izračunavanje optimalne amplitude u frekventnom domenu, čime se minimizuje kvadratna greška između čistog i filtriranog signala.
- **MMSE (Minimum Mean Square Error) spektralni estimatori:** procjenjuju amplitudni spektar čistog signala pod pretpostavkom statističkog modela šuma i signala, čime se postiže efikasnije uklanjanje šuma, naročito kod bijelog i impulsivnog šuma.
- **Spectral masking i soft-masking tehnike:** primjenjuju se maske koje selektivno smanjuju ili zadržavaju frekvencijske komponente, čime se smanjuju artefakti poput muzičkog šuma.

Mašinsko učenje i neuralne mreže

U novije vrijeme, mašinsko učenje je postalo popularno za redukciju šuma, jer algoritmi mogu naučiti kompleksne obrasce šuma i signala direktno iz podataka:

- **DNN (Deep Neural Networks):** mreže trenirane na parovima šum–čist signal mogu predvidjeti spektrogram čistog signala iz šumovitog ulaza.

- **RNN/LSTM (Rekurentne neuronske mreže):** zahvaljujući memoriji u vremenu, ove mreže bolje modeliraju nelinearne i ne-stacionarne šumove u audio signalima.
- **Convolutional Neural Networks (CNN) na spektrogramima:** koriste prostornu i frekvencijsku korelaciju u spektrogramima za uklanjanje šuma dok zadržavaju detalje signala.

Ovi algoritmi često kombinuju vremensko-frekventne transformacije (poput STFT) sa dubokim mrežama, gdje mreža uči mapiranje šumovitog spektrograma na čisti spektrogram, nakon čega se signal rekonstruiše inverznom transformacijom u vremenski domen. Takve metode su u stanju ukloniti šum višeg intenziteta i kompleksne nelinearne artefakte, ali zahtijevaju značajnu količinu podataka i računarskih resursa za treniranje.

Zaključak

U ovom radu prikazana je primjena Furijeove transformacije i njenog diskretnog oblika za analizu i redukciju šuma u digitalnim audio signalima, sa posebnim osvrtom na kratkoročnu Furijeovu transformaciju (STFT). Pokazano je da klasični FFT algoritam, iako efikasan u globalnom frekvencijskom domenu, nije dovoljan za signale sa promjenljivim ili impulsivnim šumom, dok STFT omogućava vremensko-frekventnu lokalizaciju i preciznije uklanjanje šuma korišćenjem prozornih funkcija i spektralne supstrakcije. Rad takođe ističe potencijal modernih metoda zasnovanih na mašinskom učenju za adaptivnu redukciju šuma, naglašavajući kompromis između preciznosti i složenosti implementacije, čime se pruža temelj za dalja unapređenja u obradi i poboljšanju kvaliteta audio signala.

Literatura

- [1] Filip Marić, Vesna Marinković *Konstrukcija I Analiza Algoritama*
- [2] Vladimir Zorich *Mathematical Analysis II*
- [3] Alan V. Oppenheim, Ronald W. Schaffer *Discrete-Time Signal Processing*
- [4] Richard G. Lyons *Understanding Digital Signal Processing*